Session 9: Tuples, Variables, Strings & List Operations in Python

---

Detailed Theory

---

1. Introducing Tuples

- **Tuples** are **ordered, immutable** collections in Python.
- Defined with **parentheses ( )** or simply commas.
- Unlike lists, tuples **cannot be modified** after creation.
- Syntax examples:

  python

CopyEdit

t = (1, 2, 3) t2 = 4, 5, 6 # parentheses optional single = (1,) # single-element tuple requires comma

- Useful for fixed data that should not change.

---

2. Declaring Variables

- In Python, variables are **created when assigned**; no need to declare type explicitly.
- Example:

python

CopyEdit

x = 10 name = "Komal"

- Python is dynamically typed; variable types can change during runtime.

---

3. Referencing Variables

- Variables store references to objects in memory.
- Assignment copies the reference, not the object itself.
- For mutable objects, multiple variables can point to the same object.

---

4. Assigning Multiple Values at Once

- Python supports **tuple unpacking** for multiple assignments:

  python

  CopyEdit

  a, b, c = 1, 2, 3

- You can also assign the same value to multiple variables:

  python

  CopyEdit

x = y = z = 0

- Useful for concise and readable code.

---

5. Formatting Strings

- Python provides several ways to format strings:
  - **Old-style (%) formatting:**

    python

    CopyEdit

    "Hello %s" % "Komal" "You have %d apples" % 5

- **str.format() method:**

  python

  CopyEdit

  "Hello {}".format("Komal") "Coordinates: {lat}, {lon}".format(lat=10, lon=20)

- **f-strings (Python 3.6+):**

  python

  CopyEdit

  name = "Komal" f"Hello {name}"

- f-strings are the most modern, readable, and efficient method.

---

6. Mapping Lists

- Mapping applies a function to each element of a list, creating a new list of results.
- Using map() function:

  python

  CopyEdit

  numbers = [1, 2, 3] squares = list(map(lambda x: x**2, numbers))
-

List comprehensions are often preferred:

python

CopyEdit

squares = [x**2 for x in numbers]

---

7. Joining Lists and Splitting Strings

- **Joining lists:**
  - Use the + operator to concatenate lists:

    python

CopyEdit

a = [1, 2] b = [3, 4] c = a + b # [1, 2, 3, 4]

- **Splitting strings:**
  - Use the .split() method to break a string into a list:

    python

    CopyEdit

    text = "apple,banana,orange" fruits = text.split(",") # ['apple', 'banana', 'orange']
  - By default, .split() splits on whitespace.

---

8. Historical Note on String Methods

- Early Python versions (before 2.0) had limited string methods.
- Over time, many useful methods were added like .join(), .split(), .replace(), .strip().
- This enriched string handling making Python popular for text processing.